YOLO-Based Traffic Vehicle Detection, Tracking, and Line-Crossing Analysis

Computer Vision Internship — Final Project Report

Project	Traffic Vehicle Detection and Frame-by-Frame Analysis
Domain	Computer Vision
Programming language	Python
Detection model	Ultralytics YOLO11n (yolo11n.pt)
Libraries	OpenCV, PyTorch, Ultralytics
Vehicle classes	Car, Motorcycle, Bus, Truck

Project summary. A YOLO11n-based video pipeline detects road vehicles, associates detections across frames, estimates line-crossing counts, and exports annotated video plus CSV/JSON results.

1. Introduction

This project implements a video-based computer vision pipeline for detecting and tracking road vehicles and estimating traffic flow from traffic footage. A pretrained YOLO11n model identifies vehicles in individual frames. OpenCV manages video input/output, frame processing, annotation, and the optional diagnostic segmentation view.

The segmentation view applies Gaussian blur and adaptive thresholding. It is a classical image-processing diagnostic, not the semantic detector: YOLO supplies the vehicle classes.

2. Objectives

Detect vehicles frame by frame; associate detections across frames using track IDs; estimate class-wise line-crossing counts; record frame-level metrics and detection predictions; export an annotated video and machine-readable summary; provide an optional threshold-based diagnostic view.

3. Input Video

Input file: data/traffic video.mp4. OpenCV successfully opened the video and reported 1920 × 1080 resolution at 30 FPS. The recorded run processed 9,184 frames. At 30 FPS, this corresponds to approximately 306.1 seconds (5 minutes and 6 seconds) of source footage, assuming a constant frame rate.

4. Methodology

4.1 Video acquisition. OpenCV reads the source video sequentially and obtains its dimensions and frame rate. The verified input is Full HD (1920 × 1080) at 30 FPS.

4.2 YOLO detection. The project uses the pretrained Ultralytics checkpoint yolo11n.pt, with confidence threshold 0.35 and inference image size 640. The system records bounding boxes and supported vehicle labels: car, motorcycle, bus, and truck. A confidence threshold controls which detections are retained; it does not establish accuracy by itself.

4.3 Tracking. A class-aware greedy tracker matches detections to existing tracks. A match requires the same class and either IoU ≥ 0.05 or centroid distance ≤ 90 pixels. Tracks tolerate up to 12 missed frames before removal. Unmatched detections receive new IDs. This lightweight method can experience ID switches during occlusion or close interactions.

4.4 Line crossing. A horizontal counting line is placed at 60% of the frame height. For a 1080-pixel frame, the recorded line is y = 648. When a tracked centroid changes sides, the class count is incremented, and each track ID is counted at most once. Results are estimates because missed detections, occlusion, and ID switches can affect crossing events.

4.5 Segmentation diagnostic. The optional view converts frames to grayscale, applies a 5 × 5 Gaussian blur, and then Gaussian adaptive thresholding (maximum value 255, binary threshold type, block size 21 for normal frames or 11 for very small frames, C = 5). It helps inspect local contrast and foreground/background separation but does not assign semantic vehicle classes.

5. Implementation Configuration

Setting	Value
Model checkpoint	yolo11n.pt
Confidence threshold	0.35
Inference image size	640
Input resolution	1920 × 1080
Source frame rate	30 FPS
Counting line	60% of frame height; y = 648
Tracker minimum IoU	0.05
Tracker maximum centroid distance	90 pixels
Maximum missed frames	12
Gaussian kernel	5 × 5
Adaptive threshold	Gaussian; block size 21 (or 11 for small frames); C = 5

6. Runtime Environment

Component	Recorded / reported value
CPU	Intel Core i5-9400F
GPU	NVIDIA GeForce GTX 1060 6 GB
RAM	16 GB
Operating system	Windows; exact edition/build not recorded
Python	Python 3.14.7 reported; verify active environment if reproducing
Deep-learning runtime	An earlier environment check reported CPU-only PyTorch; GPU acceleration is not claimed here
Video processing	OpenCV
Detection framework	Ultralytics YOLO

7. Experimental Results

The recorded run processed 9,184 frames in 577.106 seconds. The values below describe this run and should not be treated as universal benchmark results.

8. Performance Metrics

Metric	Result
Frames processed	9,184
Elapsed processing time	577.106 seconds
Mean inference time	41.877 ms
Median inference time	36.310 ms
95th percentile inference time	63.424 ms
Mean processing speed	15.914 FPS
Mean detections per frame	10.604
Mean CPU utilization	443.64% as reported by the process metric
Peak process memory	533.84 MB

9. Estimated Line-Crossing Counts

Vehicle class	Estimated crossings
Car	294
Motorcycle	0
Bus	27
Truck	127
Total crossing events	448

These are estimated counts, not ground-truth-validated traffic counts. The motorcycle class had 51 detection records but zero counted crossings; review the annotated video and CSV records to investigate.

10. Output Artifacts

Reported outputs are outputs/annotated_traffic.mp4 (annotated video), outputs/frame_metrics.csv (frame-level metrics), outputs/predictions.csv (detection records and tracking information), and outputs/summary.json (configuration and aggregate metrics).

The prediction export was summarized as 97,391 detection records: 85,848 car records, 9,154 truck records, 2,338 bus records, and 51 motorcycle records. These are per-frame observations, not unique physical vehicles. A single vehicle can contribute many detection records.

The recorded class crossing totals sum to 448, matching the reported 448 unique track IDs crossing the line. The motorcycle class had 51 detection records but zero counted crossings. This should be investigated by reviewing the annotated video and CSV because detections do not guarantee that a track crossed the line.

11. Limitations

No ground-truth annotations were provided for this video, so precision, recall, F1-score, and detection IoU accuracy are not reported. Line-crossing counts are estimates and may be affected by occlusion, missed detections, bounding-box jitter, and ID switches. The greedy tracker is simpler than dedicated multi-object trackers such as ByteTrack or BoT-SORT. Adaptive thresholding is a diagnostic view and may be sensitive to shadows and lighting. The measured processing speed (15.914 FPS) was below the source rate of 30 FPS, so this run did not process the video in real time. The exact Windows build and active PyTorch/CUDA configuration were not fully recorded.

12. Conclusion and Future Work

The project provides an end-to-end pipeline for YOLO-based vehicle detection, lightweight tracking, line-crossing estimates, annotated video output, and CSV/JSON logging. In the recorded run, it processed 9,184 frames at a mean processing speed of 15.914 FPS and estimated 448 line-crossing events across cars, buses, and trucks.

Future work should validate counts against manually labelled ground truth, investigate the zero motorcycle-crossing result, tune tracking and counting thresholds, compare a dedicated multi-object tracker, and benchmark a verified GPU-enabled environment. Accuracy claims should only be added after ground-truth evaluation.

13. Reproducibility

Input: data/traffic video.mp4 | Model: yolo11n.pt | Confidence: 0.35 | Image size: 640 | Counting line position: 0.60 of frame height | Output directory: outputs.

Activate the intended project environment, install the project's dependencies, and run the existing main program with the source path in quotes because the filename contains a space.

14. Metric Definitions and References

Inference time is the measured model-inference time per frame. Processing FPS is the number of frames processed per second across the run and includes more than inference. A detection record is one object detected in one frame; the same vehicle can create many records. A track ID is a temporary association and is not guaranteed to identify one physical vehicle perfectly. A line-crossing count is incremented when a tracked centroid moves across the counting line.

References: Ultralytics YOLO documentation, https://docs.ultralytics.com/ ; OpenCV documentation, https://docs.opencv.org/ ; PyTorch documentation, https://pytorch.org/docs/stable/index.html